Rapport de soutenance

EPITA Rennes

Tower Attack



Réalisé par Tower Coders

TOWER CODERS

Norah Beillevert Thomas Vigneau Raphaël Dréau Foucauld Bracq Julien Jollivet

Table des matières

1 Introduction **4**
- 1.1 Tower Coders : L'équipe . . . 4
- 1.2 Notre projet : Tower Attack . . . 7
- 1.3 Histoires . . . 8
  - 1.3.1 Histoire du jeu . . . 8
  - 1.3.2 Celestin . . . 8
- 1.4 Les Logos . . . 9

2 Les Unités **13**
- 2.1 Histoire des troupes . . . 13
- 2.2 Les Unités . . . 14
  - 2.2.1 Catégories d'unités . . . 14
  - 2.2.2 Types d'attaque : Distance et Mêlée . . . 15
- 2.3 Implémentation technique . . . 15
  - 2.3.1 Scriptable Objects : TroupeType / TowerType . . . 15
  - 2.3.2 MonoBehaviours : Troupe / Tower . . . 16
  - 2.3.3 Instanciation des troupes . . . 16
- 2.4 Déplacement des troupes . . . 17
- 2.5 Animation des unités . . . 18
- 2.6 Les tours . . . 19
  - 2.6.1 Fonctionnement technique . . . 20

3 Le jeu **24**
- 3.1 Le Menu . . . 24
- 3.2 Les intelligences artificielles . . . 27
  - 3.2.1 L'IA qui joue contre le joueur . . . 27
  - 3.2.2 l'IA qui dirige le comportement des tourelles . . . 28
  - 3.2.3 l'IA qui dirige la tours pour visé au mieux la troupe . . . 29
  - 3.2.4 Intelligence artificielle dans le mini-jeu (Bringer of Death) . . . 29
- 3.3 Le multijoueur . . . 30
  - 3.3.1 Évolution du concept . . . 30
  - 3.3.2 Concept final retenu . . . 31
  - 3.3.3 Implémentation technique . . . 31
  - 3.3.4 Synchronisation réseau . . . 32
  - 3.3.5 Conclusion . . . 32
  - 3.3.6 La scène de chargement . . . 33
- 3.4 L'interface utilisateur . . . 35
  - 3.4.1 Barre de Vie . . . 35
  - 3.4.2 Temps et Argent . . . 36
  - 3.4.3 Acheter des unités . . . 36

3.4.4 Les paramètres 37
3.4.5 Le tutoriel 37
3.4.6 Système de caméra 37
3.4.7 Conclusion 39
3.5 Les cartes 40
3.6 Choix musicaux et ambiance sonore 42
3.7 Communication 44
3.7.1 L'intérieur de l'entreprise 44
3.7.2 Instagram 44
3.7.3 Site internet 45
3.8 Conclusion 46
3.8.1 Tableau de Gantt 46
3.8.2 Avancement 46
3.8.3 Ressenti du projet 47
3.8.4 Compétences et apprentissages 49
3.8.5 Conclusion de l'entreprise 50

1 Introduction

1.1 Tower Coders : L'équipe

Tower Coders est une jeune start-up bretonne fondée par Foucauld Bracq en septembre 2024, avec pour ambition de redéfinir les standards du jeu vidéo indépendant. Plutôt que de miser sur la quantité, notre entreprise privilégie une approche centrée sur la qualité et l'innovation pour offrir des expériences de jeu captivantes et accessibles à un large public.

Accompagné par une équipe talentueuse composée de Norah, Thomas, Julien et Raphaël, Foucauld lance la première aventure de Tower Coders : Tower Attack. Contrairement au classique Tower Defense, ce jeu propose une expérience innovante, où les joueurs devront prendre l'offensive dans un gameplay inédit et palpitant.

L'équipe de cinq développeurs passionnés s'est fixé un défi de taille : finaliser le projet en moins d'un an, tout en rivalisant avec les jeux les plus connus du marché. Une mission ambitieuse, portée par la créativité, la détermination et une volonté commune de marquer l'industrie du jeu vidéo.

L'équipe Tower Coder



Foucauld — Chef du projet

Je suis développeur en herbe depuis le début du lycée et j'ai fait mes premiers pas sur ma calculatrice, où j'ai appris la quasi-intégralité des bases nécessaires pour le développement de scripts plus avancés. Une fois en terminale, je me suis construit, seul, un PC fixe pour disposer de plus de puissance et pouvoir poursuivre le développement de mon IA. Aujourd'hui, j'ai 18 ans et je m'attaque une nouvelle fois à un gros morceau, grâce à *Tower Attack* : développer un jeu vidéo dans un environnement 3D. *Tower Attack* est l'entreprise que nous avons choisie de créer pour soutenir ce projet.

Thomas — Concepteur narratif et designer



Moi, c'est Thomas Vigneau, j'ai 18 ans et je suis passionné d'informatique. J'aime les jeux vidéo et le sport (surtout les arts martiaux). Pour faire court, je suis un jeune homme dynamique, attentif et fiable. J'ai rejoint *Tower Coder* car le projet *Tower Attack* m'intéresse vraiment. Mes compétences en informatique et mon calme peuvent apporter autant au projet que ce que le projet peut m'apporter, tant en satisfaction personnelle que dans l'aventure qu'il représente. Pour ce projet, je suis chargé du storytelling, du design des personnages et de leur implémentation.

Norah — Créatrice d'univers et d'ambiance



Enchantée, je m'appelle Norah, j'ai 18 ans également. Passionnée (ou même accro) aux jeux vidéo de stratégie et d'horreur, tels que *Fear and Hunger*, je nourris un intérêt profond pour le développement de ce sur quoi j'ai passé des heures de ma vie. C'est donc pour cette raison que j'ai rejoint l'entreprise *Tower Coder* pour son merveilleux projet, *Tower Attack*. Je compte y apporter mon imagination dans la création du storytelling afin d'assurer une cohérence dans le gameplay et l'évolution du joueur, mais aussi mettre mes compétences au service du développement du graphisme et de l'ambiance du jeu, grâce à mon expérience en tant que joueuse.

Raphaël — Game designer et développeur IA



Salut tout le monde, je m'appelle Raphaël, j'ai 20 ans et je suis étudiant en école d'ingénieur informatique à l'Épita. Passionné d'informatique, de jeux vidéo et de sport, je suis une pile d'énergie, attiré par le game design. Mon rôle dans l'équipe est de concevoir le gameplay, notamment l'implémentation des unités et leurs interactions. Grâce à mes bases solides en mathématiques, je soutiens également Foucauld, notre fondateur, dans le développement de l'intelligence artificielle. Je m'investis pleinement dans *Tower Coder* et son projet phare : *Tower Attack*. Depuis mes débuts ici, j'ai trouvé bien plus qu'une équipe : une vraie famille. Ensemble, portés par notre passion, nous avons l'ambition de faire notre place dans le monde du jeu vidéo.

Julien — Développeur web et intégrateur



Alors, moi c'est Julien, j'ai 19 ans et je suis né à Paris. Je veux devenir programmeur, et c'est pour cela que je me suis lancé dans des études pour apprendre différents langages de programmation. J'ai donc intégré le lycée Marcelin Berthelot, le lycée le plus proche de chez moi, qui proposait la spécialité Numérique et Sciences de l'Informatique (NSI). Après trois années de lycée, au cours desquelles j'ai créé un jeu vidéo, je rentre à l'EPITA en septembre 2024 et rejoins *Tower Coder* à la même période pour créer des jeux vidéo de qualité. Pourquoi cette équipe ? Eh bien, pour son projet tout simplement innovant, qui prend à contre-pied le concept de *Tower Defense* : *Tower Attack*. J'aurai en charge le site web, l'ambiance du jeu, et je naviguerai entre la programmation et le design pour assurer une cohérence entre les deux.

1.2 Notre projet : Tower Attack

Découvrez un concept novateur qui revisite les mécaniques classiques des jeux de Tower Defense.

Dans *Tower Attack*, vous ne défendez plus votre base : vous passez à l'offensive ! Ce jeu de stratégie en temps réel (RTS) vous invite à planifier des attaques épiques en construisant et en améliorant vos armées. Utilisez vos ressources avec soin, coordonnez vos unités et élaborez des stratégies pour surpasser les défenses ennemies.

Mécanique de jeu :

Vous disposerez d'une zone où vous pourrez placer vos unités d'attaque, achetées avec l'argent gagné au fil de la partie. Ces unités avanceront automatiquement sur un chemin prédéfini et affronteront les défenseurs rencontrés en chemin. Si vos unités atteignent le château ennemi, celui-ci subira des dégâts. Lorsque le château aura encaissé suffisamment de dégâts, vous remporterez la victoire.

Cependant, la stratégie sera essentielle : si votre attaque manque de ruse et d'organisation, les défenses ennemies deviendront progressivement plus puissantes, rendant la victoire impossible. Dans ce cas, ce sera une défaite pour vous.

Inspiration et Univers Développé par *Tower Coder*, une entreprise passionnée par les jeux de type Tower Defense, *Tower Attack* s'inspire des classiques du genre tout en apportant une touche unique et dynamique. L'idée trouve ses racines dans des jeux cultes comme *Herzog Zwei* (1989), l'un des premiers RTS, dont l'impact continue d'influencer la scène vidéoludique.

En plus de son gameplay stratégique captivant, *Tower Attack* plonge les joueurs dans une aventure unique. Dans un univers fantastique, vous incarnez l'élu(e) chargé(e) de sauver Célestin, roi du C# et maître du code, kidnappé par le redoutable Seigneur des Bugs. Vous traverserez différentes époques et royaumes pour restaurer l'ordre et libérer Célestin afin de triompher du chaos.

Préparez-vous à repenser les codes du Tower Defense et à passer à l'attaque avec *Tower Attack* !

1.3 Histoires

1.3.1 Histoire du jeu

Dans un royaume numérique se trouvait un canard pas comme les autres. Célestin, reconnaissable par son magnifique plumage bleu et sa couronne dorée, règne sur le monde du C#. Mais ce n'était pas seulement un roi, Célestin est aussi un guide pour tous les développeurs, débuggant chaque programme grâce à son calme et à sa sagesse légendaire. Il est par ailleurs connu sous le nom de "Duck master". Dans son univers, Célestin est plus qu'un simple canard codeur, il est le roi du jeu ! Son rôle est de te faire découvrir son monde afin que tu puisses surmonter tous les obstacles. Mais hélas, alors que tu te familiarises à peine avec les commandes, prêt à partir à l'aventure, un drame s'abat sur Rutoh ! Une force maléfique, le terrible seigneur des bugs, ennemi juré de Célestin, envahit notre royaume et kidnappe notre majesté afin de l'amener dans le royaume des Glitches. Le royaume de Célestin est en ruines, désorganisé et fragmenté, privé de son roi bien-aimé... Tu es l'élu(e), celui ou celle qui devra ramener l'ordre et la paix au royaume. Ton but est clair : libérer Célestin, et pour cela il faudra parcourir les différentes principautés sous le contrôle du seigneur des bugs.

Celestin est originaire du village de Rutoh, un petit village médiéval essentiellement peuplé de paysans. Ce village a pris de l'ampleur depuis que celestin est devenu roi. Celestin s'était entouré d'une garde très compétente pour le protéger. Ses deux gardes étaient Brutus le barbare et Aerin l'elfe, les meilleurs guerriers de leurs peuples respectifs. Mais cela n'a pas suffi. Afin de libérer le roi, les barbares et les elfes, les alliés de Celestin, sont à votre disposition.

1.3.2 Celestin

Le choix d'un canard comme personnage principal n'est pas anodin. Dans le monde de la programmation, le "rubber duck debugging" (ou "débogage par le canard en plastique") est une méthode bien connue : elle consiste à expliquer son code, ligne par ligne, à un objet inanimé, souvent un canard, afin de repérer plus facilement les erreurs logiques ou les oublis. C'est une technique simple mais redoutablement efficace, utilisée par de nombreux développeurs pour résoudre des bugs. Nous avons voulu donner vie à ce compagnon silencieux et en faire un véritable héros numérique. Célestin incarne ce canard : sage, calme, et toujours prêt à aider à corriger les erreurs. Le hisser au rang de roi était une manière symbolique de souligner son importance capitale dans le monde du développement. Il n'est plus seulement un outil, mais un guide, un leader, une figure emblématique.

Son plumage bleu n'est pas un hasard non plus : le bleu est la couleur associée à la technologie, au numérique et à la fiabilité. C'est aussi une couleur rassurante, qui renforce le caractère bienveillant et posé de notre "Duck Master".

1.4 Les Logos

Le travail de création des logos a été mené en collaboration par **Philippine Braqc**, **Norah** et **Chat GPT**. Nous avons réfléchi ensemble à l'identité visuelle de l'équipe de développement (*Tower Coders*) ainsi qu'à celle du jeu (*Tower Attack*), en veillant à créer une cohérence graphique entre les deux tout en transmettant les valeurs et l'univers de notre projet.

1. L'entreprise : Tower Coders



Le nom

Le nom **Tower Coders** s'inspire directement du genre de jeu vidéo bien connu des développeurs : les *Tower Defense*. Dans ces jeux, les *towers* (tours) désignent des unités défensives que le joueur place stratégiquement pour stopper des vagues ennemies. Le mot **Tower** ancre donc immédiatement le projet dans l'univers du **game design stratégique**, sans nécessairement faire référence à un imaginaire médiéval.

L'association avec le mot **Coders** reflète la nature de l'équipe : des développeurs passionnés, créateurs de jeux et d'expériences interactives. Ce nom évoque l'idée d'une **communauté de développeurs spécialisés dans les jeux de stratégie**, unis par leur capacité à bâtir des mécaniques solides comme des tours.

Le logo

Le logo de **Tower Coders** a été pensé pour être **simple, polyvalent et immédiatement identifiable**.

- La **tour stylisée** rappelle visuellement une tour de jeu (et peut évoquer à la fois une *tour de défense* ou une *pièce d'échiquier*, pour la stratégie).
- Le logo est initialement conçu en **bleu**, une couleur :
  - Associée à la **technologie** et à la **confiance**,
  - Rappelant également les couleurs courantes dans l'univers de la programmation (interfaces, IDE...).
- Cependant, afin de garantir une **bonne lisibilité sur tous supports**, le logo est conçu pour être **déclinable** :
  - En noir et blanc pour les supports neutres,
  - Ou en s'adaptant à des fonds variés (sombres, clairs, texturés).
- La typographie utilisée est :
  - **Sans empattement**, pour renforcer l'aspect **moderne et technique**,
  - En **lettres capitales**, évoquant la **solidité** et le **sérieux professionnel**.

Ce logo agit donc comme un **label** de qualité technique, incarnant une équipe capable de coder des jeux robustes, bien pensés, et ancrés dans des genres exigeants.

2. Le jeu : Tower Attack



Le concept

Notre jeu **Tower Attack** est un **RTS** (jeu de stratégie en temps réel) s'inspirant fortement des jeux **Tower Defense**. Dans un Tower Defense classique, le joueur incarne une base attaquée par des vagues d'ennemis générées automatiquement. Le but est alors de placer stratégiquement des tours afin de repousser l'ennemi jusqu'à ce que toutes les vagues soient vaincues ou que le temps imparti s'écoule.

Dans **Tower Attack**, nous avons inversé cette logique. Ici, **c'est le joueur qui passe à l'offensive** : entre chaque manche, il constitue son armée en achetant des unités à l'aide des ressources accumulées, puis les envoie attaquer une base ennemie contrôlée par une **IA défensive**. Cette IA agit comme le joueur d'un Tower Defense classique : elle place automatiquement des tours pour défendre son château. L'objectif du joueur est alors de **réduire les points de vie du château ennemi à zéro** pour remporter la partie.

Le logo

Le logo de **Tower Attack** a été conçu pour transmettre immédiatement l'univers **épique, dynamique et stratégique** du jeu.

1. Les éléments visuels

- La **tour grise** en haut du mot "TOWER" rappelle le logo de l'entreprise, créant une **continuité visuelle** entre la société et son produit.
- L'**épée dorée** traversant le logo représente :
  - L'**agressivité maîtrisée**, le combat, et l'aspect **offensif** du gameplay.
  - Une séparation symbolique entre les deux mots : **TOWER** (défense) et **ATTACK** (offensive).

2. Typographie

- La typographie est **massive, avec empattements**, évoquant la **force**, la **puissance** et une ambiance **fantasy**.
- Les **lettres dorées** pour "TOWER" donnent une impression de **prestige et de noblesse**, faisant allusion à un château ou un royaume.
- Les **lettres bleues** pour "ATTACK" suggèrent :
  - La **froidure du combat**, mais aussi
  - La **couleur du numérique**, renforçant l'aspect tech/fantastique du jeu.

3. Couleurs

- **Or (doré)** :
  - Évoque la **richesse, le pouvoir royal**, la noblesse du personnage principal.
- **Bleu** :
  - Couleur souvent associée à **la technologie**, **l'intelligence**, mais aussi à **l'aventure et au froid**.
  - Elle contraste avec l'or, soulignant l'opposition thématique entre **défense/attaqu**
    royaume/ennemi, **ordre/chaos**.
- **Gris foncé** pour la tour :
  - Suggère la **solidité**, la **neutralité**, et fait un lien visuel avec le logo de Tower Coders.

Conclusion

Ces deux logos ont été pensés pour créer une **cohérence visuelle entre l'identité de l'équipe (Tower Coders)** et celle du **jeu (Tower Attack)**. On retrouve un univers graphique qui allie **technologie, stratégie et aventure**, en phase avec l'identité du projet : une production sérieuse, moderne, et ancrée dans une culture du code créatif.

2 Les Unités

2.1 Histoire des troupes



Brutus — Le Barbare Loyal

Brutus est un barbare brutal et loyal, servant comme garde du corps personnel de Célestin. Impulsif et violent en combat, il devient calme et protecteur en présence de son maître. Il manie sa hache avec une force dévastatrice, capable de briser les lignes ennemies d'un seul coup.



Aerin — L'Ombre Silencieuse

Aerin est une archère d'exception, réputée pour ne jamais manquer sa cible. Silencieuse et observatrice, elle veille sur Célestin depuis les hauteurs, prête à abattre quiconque oserait le menacer. Sa précision redoutable en fait un atout vital de la garde rapprochée.



Balthazar — Le Sage Arcanique

Balthazar est un mage légendaire, si ancien que certains disent qu'il aurait formé Merlin lui-même. Alliant puissance magique, savoir ancien et un humour cinglant, il protège Célestin au sein de sa garde rapprochée avec des sorts aussi puissants qu'imprévisibles.



Perceval — Le Chevalier Vengeur

Perceval est le champion de Célestin, son plus puissant chevalier. Absent lors de l'enlèvement de son maître, il revient de mission avec une seule obsession : libérer Célestin, quel qu'en soit le prix.



Igor le Maudit — L'Enigmatique Allié

Igor est un être surnaturel dont on ne connaît ni son espèce, ni ses origines. Une chose est sûre : il voue une haine inconditionnelle au seigneur des bugs. C'est pourquoi il a proposé son aide à nos héros.

2.2 Les Unités

Dans notre jeu, les unités sont des éléments centraux et se divisent en deux grandes catégories : les tourelles (défensives) et les troupes (offensives). Ces entités sont représentées chacune par une classe distincte et disposent de caractéristiques propres. Leur gestion repose sur une architecture orientée objet facilitant la modularité et l'extensibilité.

2.2.1 Catégories d'unités

— **Tourelles (Towers)** : Utilisées par l'IA pour se défendre, elles attaquent automatiquement les troupes adverses en fonction de règles de priorité.

— **Troupes (Troupes)** : Contrôlées par le joueur, elles progressent sur un chemin prédéfini pour attaquer la base adverse.

La principale différence fonctionnelle réside dans l'attribut de vitesse, spécifique aux troupes, qui leur permet de se déplacer vers l'objectif.

2.2.2 Types d'attaque : Distance et Mêlée

Les troupes ne se battent pas toutes de la même façon. On distingue deux grands types d'unités offensives selon leur style d'attaque :

- **Unités à distance** : Aerin et Balthazar peuvent attaquer de loin.
  - Aerin tire une *flèche* précise et rapide qui se dirige directement vers la cible.
  - Balthazar lance une *boule de feu* magique, visuellement identifiable, avec des effets puissants.

Ces unités instancient des projectiles au moment de l'attaque, ce qui permet de toucher les ennemis sans contact direct.

- **Unités de mêlée** : Brutus, Perceval et Igor doivent s'approcher physiquement de l'ennemi pour infliger leurs dégâts. Ils ne produisent pas de projectiles mais frappent directement la cible à proximité.

Cette distinction stratégique offre au joueur un large éventail de possibilités tactiques : combiner puissance brute à courte portée avec des frappes précises à distance.

2.3 Implémentation technique

2.3.1 Scriptable Objects : TroupeType / TowerType

Les unités sont paramétrées grâce à des Scriptable Objects, développés par **Foucauld** et **Norah**, qui définissent leurs attributs personnalisables :

- Vie
- Dégâts
- Vitesse d'attaque
- Portée
- Prix
- Vitesse (uniquement pour les troupes)

Ces objets permettent de créer différents types d'unités avec des comportements cohérents et spécifiques selon les données renseignées par l'équipe Game Design (Thomas et Norah).

2.3.2 MonoBehaviours : Troupe / Tower

Chaque unité instanciée dans la scène hérite d'une classe MonoBehaviour (`Troupe.cs`, `Tower.cs`) qui référence son Scriptable Object respectif. Ces scripts implémentent les comportements dynamiques tels que :

— Affichage de la portée en temps réel via `OnDrawGizmos`.
— Gestion du comportement d'attaque.
— Affichage de la barre de vie proportionnelle à la vie maximale.

2.3.3 Instanciation des troupes

Lorsqu'un joueur appuie sur un bouton d'interface correspondant à une unité, une troupe est immédiatement instanciée sur le terrain, dans la zone de déploiement prévue à cet effet. Chaque troupe reçoit automatiquement un nom unique, construit à partir d'un compteur global incrémenté à chaque invocation (ex. : Archer_003, Barbare_014), ce qui facilite leur identification et leur gestion dans le jeu.

L'apparition de la troupe est instantanée, à la fois visuellement dans la scène et dans les systèmes internes du jeu, ce qui permet une réaction rapide et fluide du gameplay.

L'interface de sélection des troupes a été enrichie depuis sa version initiale. Elle affiche désormais :

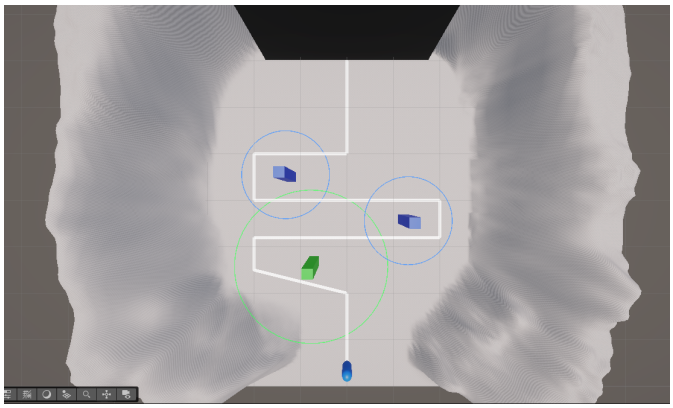
— Le coût de chaque unité en ressources, permettant au joueur d'anticiper ses choix en fonction de son budget actuel
— Une liste dynamique des troupes déjà invoquées, avec leur nom et leur état (vivante, en combat, éliminée)
— Une indication visuelle de l'accessibilité des unités, grisée si le joueur ne peut pas se la permettre pour le moment

Chaque bouton de l'interface est lié à une fonction d'invocation sécurisée, qui vérifie automatiquement si le joueur possède les ressources nécessaires avant de valider l'appel de la troupe. Cela évite les erreurs de clic et renforce le lien entre économie et stratégie.

Ce système d'instanciation est central dans la boucle de gameplay, puisqu'il permet au joueur de gérer activement ses troupes et de s'adapter en temps réel aux décisions de l'IA ennemie.

2.4 Déplacement des troupes

Les troupes apparaissent dans une zone de départ puis avancent automatiquement vers la base ennemie en suivant un chemin prédéfini. Ce chemin, conçu par **Foucauld**, est défini par une série de points formant une trajectoire que chaque unité doit suivre, comme une sorte de ligne de guidage à travers la carte.



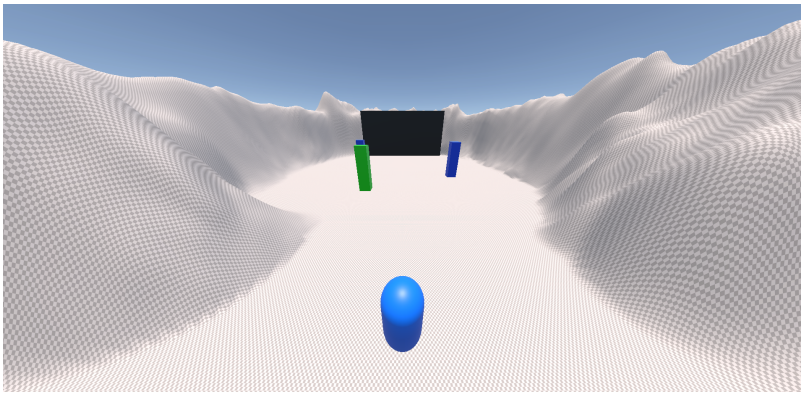
À chaque instant, la troupe se dirige vers le point suivant sur le trajet. Lorsqu'elle l'atteint, elle se tourne vers le suivant, jusqu'à atteindre l'objectif final. Ce système assure que toutes les unités avancent dans la bonne direction, tout en respectant leur propre vitesse de déplacement. Le mouvement est mis à jour en continu à chaque image du jeu, ce qui permet une progression fluide.

Des effets extérieurs peuvent influencer cette progression : par exemple, une troupe peut être ralentie par un piège ou accélérée par un sort. Ces modifications sont prises en compte dynamiquement pendant son trajet.

Afin de rendre le déplacement plus agréable visuellement, nous avons mis en place un système de transition douce entre les changements de direction. Cela évite que les troupes tournent brusquement et améliore l'impression de fluidité, surtout dans les virages.

De plus, trois caméras sont attachées à chaque troupe : une vue de dessus, une vue de côté et une vue subjective. Cela permet de suivre leurs déplacements sous différents angles et d'enrichir l'expérience de jeu.

Enfin, une amélioration en cours d'étude consiste à remplacer les lignes droites du parcours par des courbes de Bézier. Ces courbes permettraient aux troupes de se déplacer de manière plus naturelle, notamment dans les zones sinueuses. Bien que leur mise en œuvre soit plus complexe, car elle rendent impossible le fait d'avancer à vitesse constante, elles offrent un rendu visuellement plus réaliste.



2.5 Animation des unités

Dans *Tower Attack*, l'animation des unités joue un rôle fondamental dans la lisibilité et l'immersion du jeu. Chaque personnage est animé en 3D par **Thomas** à l'aide d'un système d'Animator Controller configuré dans Unity.

Ce système permet d'enchaîner plusieurs états visuels — *repos*, *déplacement*, *attaque*, *mort* — en fonction du comportement de l'unité dans le jeu. Les animations sont issues de *Mixamo* ou de l'*Unity Asset Store*, puis intégrées et affinées pour correspondre au style et au rythme du gameplay.

Le système repose sur des paramètres comme `isRunning`, `isAttacking`, `isDead`, etc., qui sont mis à jour par un script C# à chaque frame en fonction de l'état logique de l'unité.

Ces booléens déclenchent les transitions dans l'Animator. On utilise également des *Blend Trees* pour gérer de manière fluide le passage entre la marche et la course selon la vitesse réelle, ainsi que des conditions de transition pour que les animations ne s'interrompent jamais brutalement.

Les animations de mort, en particulier, sont volontairement sobres et rapides : elles ne cherchent pas à impressionner, mais à indiquer clairement qu'une unité a quitté le champ de bataille.

Ce système permet à toutes les unités de répondre de façon cohérente aux événements du jeu : elles se déplacent de manière fluide, s'arrêtent pour attaquer, puis reviennent à l'état d'attente.

Lorsque leur vie atteint zéro, elles jouent une animation de chute simple et claire avant de disparaître ou d'être désactivées.

Même si toutes les unités partagent la même base structurelle dans leur Animator, chacune conserve une gestuelle distincte qui reflète son rôle :

L'archère est rapide et agile, avec des animations légères pour le déplacement et une position bien ancrée au moment du tir. Elle peut attaquer de loin et au corps à corps.

Le **barbare** est plus massif, avec des gestes amples et puissants qui rendent ses attaques particulièrement visibles.

Le **mage** conserve un style mesuré : ses mouvements sont calmes, et ses animations d'attaque font appel à des gestes amples mais contenus, évoquant la concentration magique.

Le **paladin** adopte une posture droite et disciplinée, alternant entre mouvement défensif et frappe contrôlée, toujours avec une certaine lourdeur.

Enfin, le **vampire** reste assez neutre dans sa démarche et dans ses attaques. Il se distingue moins par l'exagération que par une sobriété inquiétante, renforçant son rôle mystérieux.

L'un des points forts de ce système est sa cohérence. Grâce à un contrôle centralisé par script et à une structure d'Animator uniforme, il est facile d'ajouter de nouvelles unités ou de modifier un comportement d'animation existant.

Ce design permet une grande souplesse de développement tout en maintenant une forte homogénéité visuelle. L'ensemble donne aux affrontements un rythme fluide et agréable, où chaque action reste lisible, même au cœur des combats les plus intenses.

2.6 Les tours

Le jeu comporte **quatre tours**, réparties en deux catégories : **classiques** et **Tesla**.

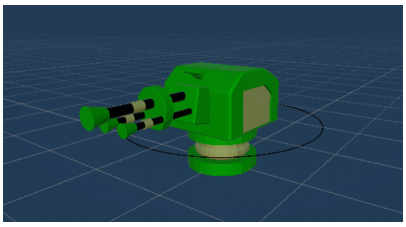
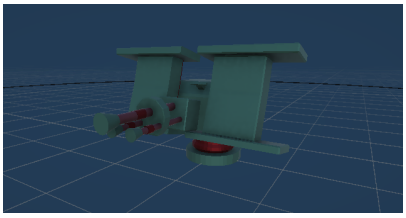
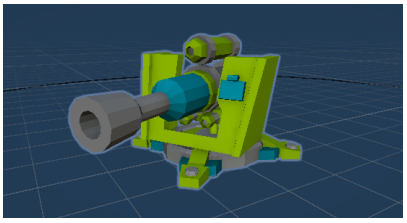
Chaque tourelle possède :

— Un modèle 3D avec animation ;
— Des caractéristiques spécifiques (portée, cadence, puissance, etc.) ;
— Un comportement d'attaque propre.

Tourelles classiques — Ces tourelles tirent des projectiles sur les ennemis selon huit priorités configurables :

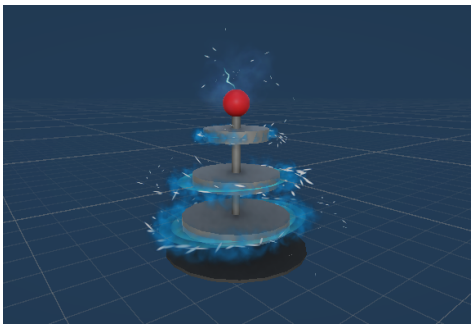
`First`, `Last`, `Strongest`, `Weakest`, `Closest`, `Furthest`, `Healthiest`, `Unhealthiest`

Leur canon s'oriente dynamiquement vers la cible en temps réel, offrant une animation fluide et cohérente avec l'action.

Tourelle Tesla — Cette tourelle attaque simultanément tous les ennemis dans sa portée, générant un rayon électrique individuel pour chaque cible.

Elle inflige des dégâts en chaîne tout en gardant une lisibilité claire, grâce à une animation rapide et localisée.



2.6.1 Fonctionnement technique

LES TOURELLES

Fonctionnement général

La tourelle, développée par **Thomas**, fonctionne elle aussi de manière autonome, dès qu'elle est placée sur le champ de bataille. Contrairement à la Tesla qui attaque tous les ennemis dans sa portée, la tourelle concentre sa puissance de feu sur une seule cible à la fois. Elle commence par analyser les troupes ennemies à proximité, et choisit la plus pertinente selon une logique de priorité (comme la plus proche, la plus dangereuse ou la plus avancée). Une fois une cible choisie, la tourelle oriente sa tête de tir mécaniquement dans sa direction.

Cette rotation est fluide et constante, ce qui permet à la tourelle de suivre sa cible même en mouvement. Lorsque celle-ci est bien alignée et que le temps de rechargement est terminé, la tourelle tire automatiquement un projectile ("bullet") qui se dirige droit vers l'ennemi. Ce projectile est instancié dynamiquement à chaque tir, et se déplace en direction de la cible sélectionnée, reproduisant un comportement fidèle aux tourelles fonctionnelles utilisées en multijoueur.

En multijoueur, cette action est gérée de manière centralisée par l'hôte (le Master Client), ce qui garantit une synchronisation parfaite entre les joueurs. En solo, le comportement est identique mais géré localement.

Utilité stratégique

L'efficacité de la tourelle réside dans sa capacité à éliminer rapidement les ennemis isolés ou résistants. Plutôt que de disperser ses attaques, elle concentre ses tirs sur une cible unique jusqu'à sa destruction, ce qui la rend redoutable contre les troupes plus robustes ou les ennemis prioritaires.

Elle s'intègre donc parfaitement dans une stratégie défensive complémentaire : là où la Tesla affaiblit ou ralentit les groupes, la tourelle termine le travail avec précision. Elle est particulièrement utile lorsqu'il faut défendre un passage étroit, ou stopper un assaillant déjà bien avancé sur le champ de bataille.

Impact visuel et immersion

Visuellement, la tourelle est facilement identifiable par sa structure rigide et sa tête mobile. Lorsqu'elle vise une cible, on peut observer la rotation mécanique de sa tête, ce qui donne au joueur un retour clair sur son comportement. Lors du tir, un projectile est généré et propulsé vers la cible, renforçant le sentiment de précision et de contrôle. Le mouvement du projectile contribue à l'immersion, en donnant une lecture immédiate de l'action en cours.

Son design est sobre, fonctionnel, et évoque une technologie militaire automatisée. Elle s'insère ainsi parfaitement dans l'univers visuel du jeu, avec une cohérence esthétique forte et une lisibilité immédiate sur son rôle.

Conclusion

La tourelle est une tour défensive classique, mais essentielle. Elle ne cherche pas à attaquer plusieurs ennemis, mais à éliminer méthodiquement les menaces les plus pressantes. Fiable, précise et rapide, elle offre un parfait équilibre entre contrôle et puissance ciblée. Son comportement clair, l'animation fluide de sa tête, et le spawn visible de projectiles renforcent sa lisibilité et son utilité dans tous les types de configurations stratégiques.

LA TESLA

Fonctionnement général

La tour Tesla, développée par **Thomas**, fonctionne de manière entièrement automatique. Une fois placée sur le champ de bataille, elle détecte en permanence toutes les troupes ennemies qui entrent dans sa zone d'effet. Dès qu'une troupe est repérée, la tour émet un rayon électrique en direction de cette cible. Contrairement à d'autres tours qui ne peuvent attaquer qu'un seul ennemi à la fois, la tour Tesla a la capacité d'attaquer plusieurs cibles simultanément. Chaque ennemi présent dans sa portée est relié à la tour par un éclair visuel, créant l'illusion d'une décharge continue, comme si l'électricité circulait à travers l'air pour neutraliser tout ce qui s'approche.

Techniquement, ce rayon est matérialisé par un élément graphique appelé *Line Renderer*, qui trace une ligne brillante entre la tour et l'ennemi. Un effet de particules, représentant un impact de foudre, est également appliqué sur la cible pour renforcer l'impression de puissance. Tant que l'ennemi reste dans la portée de la tour, le rayon reste actif et se met à jour en temps réel pour suivre ses mouvements. Dès que l'ennemi est éliminé ou quitte la zone, l'effet est supprimé automatiquement, ce qui permet à la tour de se recentrer uniquement sur les menaces actives.

Utilité stratégique

L'intérêt principal de la tour Tesla réside dans sa capacité à gérer plusieurs ennemis à la fois. Lorsqu'un groupe de troupes avance en formation compacte, les tours classiques peuvent se retrouver dépassées, attaquant une seule cible pendant que les autres continuent leur progression. La Tesla, elle, offre une réponse immédiate à ce problème en attaquant l'ensemble du groupe sans discrimination. Elle est donc particulièrement efficace contre les vagues nombreuses ou les ennemis à faible résistance mais en grande quantité.

De plus, son attaque continue — sans phase de rechargement ni animation de tir — permet d'infliger des dégâts constants et réguliers. Cette régularité en fait une tour idéale à placer près des zones stratégiques, telles que l'entrée du champ de bataille ou les points proches de la base, où un grand nombre de troupes risquent de se concentrer.

Impact visuel et immersion

Sur le plan visuel, la tour Tesla est conçue pour être immédiatement identifiable. Son apparence, composée de plusieurs disques empilés et surmontés d'une sphère rouge, rappelle l'esthétique des machines électriques futuristes. Lorsqu'elle entre en action, elle produit une série d'effets lumineux dynamiques, des éclairs bleutés reliant la tour aux ennemis, renforçant l'idée de puissance et de danger.

Ces effets ne sont pas uniquement décoratifs : ils participent à l'immersion du joueur en traduisant visuellement l'impact réel de la tour sur le champ de bataille.

Conclusion

En résumé, la tour Tesla est une tour défensive à la fois redoutable et spectaculaire. Grâce à sa capacité unique d'attaquer toutes les troupes dans sa portée en continu, elle joue un rôle crucial dans la stratégie du joueur face aux vagues massives. Elle combine efficacité, simplicité d'utilisation et effets visuels impressionnants, ce qui en fait un atout incontournable dans toute partie de *Tower Attack*.

Conclusion

Les unités de Tower Attack incarnent l'essence même du jeu, à la croisée de la narration et des mécaniques stratégiques. Troupes héroïques et tourelles défensives se complètent pour offrir un gameplay riche, où chaque entité joue un rôle déterminant. Leur conception repose sur une dualité forte : d'un côté, une dimension narrative marquée à travers des personnages comme Brutus ou Aerin ; de l'autre, une architecture logicielle rigoureuse, pensée pour la modularité.

Cette structure permet une réelle profondeur stratégique. Le joueur doit jongler entre unités de mêlée et unités à distance, adapter ses choix au rythme des vagues ennemies, et composer des armées cohérentes. La séparation entre unités offensives et défensives génère une dynamique asymétrique qui stimule la prise de décision en temps réel.

Techniquement, l'usage des Scriptable Objects pour définir les caractéristiques des unités rend leur personnalisation intuitive et flexible. Couplés à des scripts MonoBehaviour spécifiques, ils assurent un comportement autonome et cohérent en jeu. L'instanciation dynamique des troupes depuis l'interface, combinée à un système de nommage automatique, permet une gestion fluide et claire de l'action.

Les tourelles, quant à elles, apportent un contrepoids stratégique fort. Leur comportement, qu'il soit ciblé ou de zone comme la Tesla, offre au joueur des solutions défensives variées et lisibles, renforçant le sentiment de contrôle sur le champ de bataille.

3 Le jeu

3.1 Le Menu

Le Menu Principal

Dès la dernière soutenance, nous avions amorcé la conception du menu principal ainsi que des premières interfaces utilisateur. Pour poser les bases visuelles, nous nous sommes appuyés sur des maquettes générées à l'aide de l'outil d'intelligence artificielle FreePik. Bien que ces modèles nous aient aidés à réfléchir à l'agencement global des éléments à l'écran, leur faible qualité d'image ne permettait pas une utilisation directe dans le jeu. Cela nous a conduit à créer nous-mêmes tous les éléments graphiques nécessaires, en veillant à leur cohérence avec l'univers du jeu.

Nous avons ainsi opté pour un design aux teintes naturelles, inspiré de la première carte du jeu se déroulant dans une forêt. Ce choix esthétique établit une ambiance calme et organique, reflétant bien le contraste du jeu entre sérénité apparente et tensions soudaines.

Depuis cette première ébauche, de nombreuses fonctionnalités ont été intégrées dans le menu principal, que nous présentons ci-dessous.

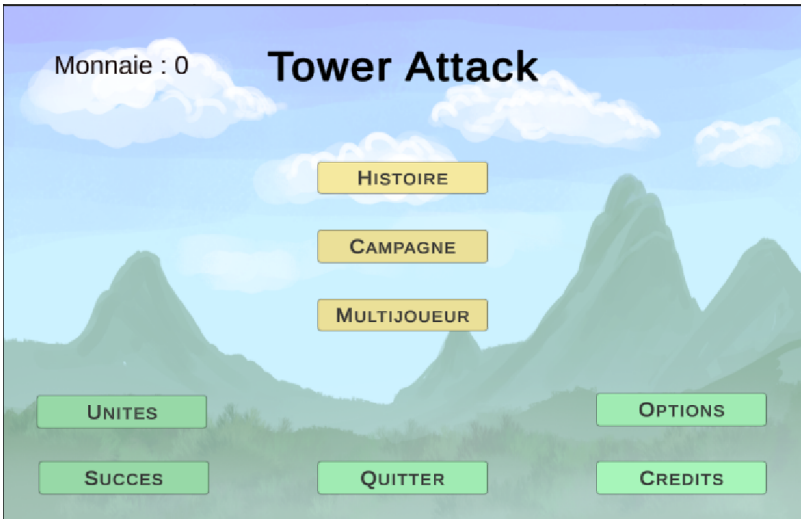


Figure 1 – Menu principal du jeu Tower Attack

Éléments du Menu Principal

Développé par **Norah**, le menu principal a été pensé pour être à la fois clair, fonctionnel et immersif. Il constitue le premier contact du joueur avec le jeu et centralise l'accès aux différentes fonctionnalités essentielles.

- **Histoire** : Affiche un texte déroulant présentant le récit du jeu. Deux boutons sont présents : "Retour" menant au Menu et "Jouer", ce dernier lançant directement la première carte.
- **Campagne** : Permet de choisir parmi les trois cartes disponibles pour jouer.
- **Multijoueur** : Permet de rejoindre un serveur existant ou d'en créer un nouveau en saisissant un nom de salle. Celui ci étant développé par **Raphael**.
- **Paramètre** : Permet de régler le volume et de mettre ou de retirer le plein écran.
- **Unités** : Accès à une boutique où le joueur peut consulter, acheter, et organiser ses unités.
- **Succès** : Affiche les succès débloqués en jeu et leur progression.
- **Quitter** : Ferme le jeu.

Interface et Sauvegarde

- En **haut à gauche** : L'affichage de la monnaie. Cette donnée est sauvegardée entre les sessions.
- En **bas à droite** : Un Bouton **Crédits**, qui affiche un texte défilant contenant les noms des membres du groupe, les remerciements, les musiques et assets utilisés, à la manière d'un générique de film.Un Bouton d'accès aux **Paramètres**, où les réglages sont également sauvegardés d'une scène à l'autre.
- En **bas à gauche** : L'accès direct aux **Succès** et au menu **Unités**.

Paramètre

Dans les paramètres, les éléments suivants sont disponibles :

- **Slider pour le volume** de la musique.
- **Bouton plein écran** (toggle) pour activer ou désactiver le mode plein écran.

Le Menu Unités

Il s'agit d'une boutique d'unités où l'on retrouve :

— Affichage de la monnaie (en haut à droite).
— Deux flèches de navigation < et > pour parcourir les différentes unités.
— Pour chaque unité : son nom, son image, sa description, et ses statistiques.
— Si l'unité est non achetée : un bouton "Acheter" s'affiche.
— Si elle est achetée : un bouton "Ajouter à l'équipe" ou "Retirer de l'équipe" est disponible. Le joueur peut constituer une équipe de **trois unités** maximum, qui seront ensuite disponibles en jeu.
— Trois unités sont disponibles gratuitement et ajoutées dans l'équipe au début.

Succès

Le système de succès est également codé et fonctionnel. Il enregistre la progression du joueur et affiche l'état d'accomplissement sous forme de texte et de barres de progression.

Voici les succès actuellement disponibles :

— **Sauver Célestin** : Terminer les 3 niveaux du mode histoire.
— **Devenir riche** : Atteindre 10 000 pièces d'or.
— **Napoléon** : Gagner 10 parties.(Nous le mettrons sûrement à une partie pour que vous puissiez tester)
— **Amitié** : Remporter une partie multijoueur.

Chaque succès est lié à une variable sauvegardée dans un fichier JSON. Lorsqu'une condition est remplie, un appel à une fonction de mise à jour est effectué. Les données sont persistantes et réutilisables à chaque démarrage du jeu.

Un exemple d'affichage dans l'interface utilisateur :

— `Sauver Célestin : 66%`
— `Devenir riche : 40%`
— `Napoléon : 80%`
— `Amitié : Non débloqué`

3.2 Les intelligences artificielles

Foucauld a développé les fonctionnalités que nous souhaitions développer en ce qui concerne les intelligences artificielles (IA) que nous avions prévus initialement.

Notre objectif était que l'IA principale remplace l'humain qui aurai joué contre nous si nous avions fait un tower défense.

3.2.1 L'IA qui joue contre le joueur

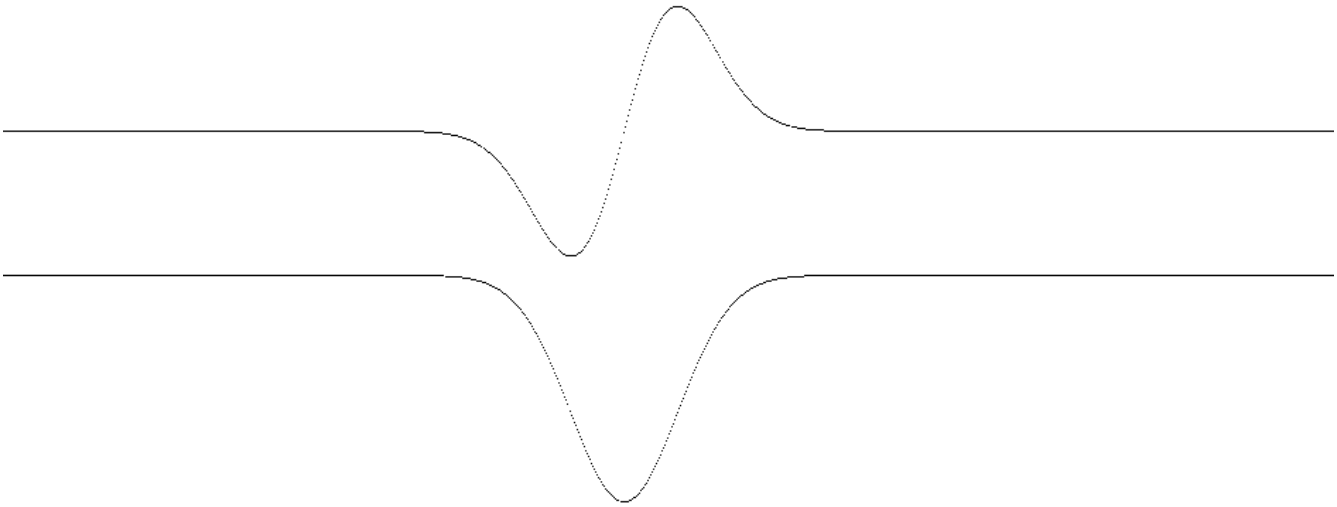
L'IA principale est celle qui remplace un joueur humain dans les choix stratégiques de défense. C'est contre elle que le joueur affronte directement, et elle est représentée dans le jeu par le personnage du Seigneur des Bugs. Cette IA a été l'une des plus complexes à concevoir, car elle doit prendre en charge toutes les décisions qu'un joueur de tower defense ferait : où placer les tours, quand les acheter, et lesquelles choisir.

Pour rendre l'expérience plus dynamique et adaptée au niveau du joueur, nous avons intégré un système de difficulté. À son niveau le plus bas, l'IA place ses tours presque au hasard, offrant peu de résistance. À l'inverse, au niveau de difficulté le plus élevé, elle analyse la situation de manière plus intelligente et cherche à optimiser ses choix pour ralentir ou éliminer les troupes du joueur le plus efficacement possible.

L'IA doit aussi gérer son économie : elle dispose d'un montant d'argent limité, comme un joueur humain, et doit donc choisir le bon moment pour investir dans de nouvelles tours. Elle ne peut pas acheter tout ce qu'elle veut immédiatement, ce qui l'oblige à faire des choix tactiques.

Lorsqu'elle place une tour, elle essaie d'identifier les zones sensibles du terrain — là où les troupes ennemies sont les plus menaçantes. L'objectif est de placer les tours au bon moment : si une tour est placée trop tard, elle n'aura pas le temps d'agir efficacement ; placée trop tôt, elle pourra anticiper l'arrivée des troupes et commencer à les viser dès qu'elles entrent dans sa portée. Ce choix du bon timing est essentiel pour que l'IA soit compétitive et crédible face au joueur.

L'image ci-dessous est sorite de unity et recoloré pour une meilleur visibilité, vous pouvez voir une troupe sur le chemin



En résumé, cette IA incarne un véritable joueur virtuel, capable de défendre sa base en prenant des décisions stratégiques en temps réel, tout en respectant les mêmes contraintes que le joueur humain.

3.2.2 l'IA qui dirige le comportement des tourelles

Au début, les tourelles ont 8 comportements différents régissants le choix de la troupe a viser. les troupes prisent pour cible seront dans la portée de la tour qui vise.

La troupe visé sera celle :

- First : avec le plus d'avance
- Last : avec le moins d'avance
- Weakest : qui fait le moins de degats
- Strongest : qui fait le plus de degats
- UnHealthiest : qui a le moins de vie restante
- Healthiest : qui a le plus de vie restante
- Closest : qui est physiquement la plus proche de la tour
- Furthest : qui est physiquement la plus loin de la tour et toujours dans la porté de la tour

Les tours changent dynamiquement de comportement en fonction des différentes troupes a l'intérieur et a l'extérieur de sa porté, la vie restante de la tour, les différentes tours déjà présentes et leurs comportements.

3.2.3 l'IA qui dirige la tours pour visé au mieux la troupe

Les tourelles visaient directement les troupes qu'elles avaient sélectionnées. Le problème était que, le temps que le projectile atteigne sa cible, celle-ci avait souvent continué à avancer, rendant ainsi possible l'esquive des tirs. Une IA a donc été développée par **Foucauld** pour améliorer la précision en anticipant la position de la cible au moment de l'impact. Cette IA prédit la position future de la troupe en fonction du temps de trajet du projectile. Elle prend en compte plusieurs paramètres : la vitesse de rotation de la tourelle, la vitesse du projectile, la vitesse de déplacement de la troupe, sa position actuelle par rapport à la tourelle, ainsi que la forme du chemin que la troupe suit après sa position actuelle.

3.2.4 Intelligence artificielle dans le mini-jeu (Bringer of Death)

L'intelligence artificielle de l'ennemi dans le mini-jeu a été conçue pour rester simple, réactive et efficace. Elle repose sur deux scripts principaux : `EnemySpawner` (gestionnaire d'apparition) et `EnemyAIParametre` (comportement de l'ennemi).

1. Système d'apparition — Le script `EnemySpawner` veille à ce qu'un seul ennemi soit actif à la fois. Il choisit aléatoirement un côté d'apparition (gauche ou droite du joueur) à une distance fixe, puis utilise un raycast pour détecter le sol et placer l'ennemi correctement. Ce dernier reçoit une référence vers le joueur afin de pouvoir l'attaquer.

2. Comportement de l'ennemi — Le script `EnemyAIParametre` gère les déplacements, attaques, dégâts et animations.

Principales fonctionnalités :

- **Suivi du joueur** : si le joueur est à portée (`chaseRange`), l'ennemi avance horizontalement grâce à la vélocité du `Rigidbody2D`.
- **Attaque** : à courte distance (`attackRange`), il s'arrête et déclenche une animation d'attaque, avec un temps de recharge pour éviter les frappes répétées.
- **Destruction** : si le joueur s'éloigne trop, l'ennemi se détruit automatiquement pour préserver les ressources.
- **Dégâts** : les points de vie diminuent lors des impacts, avec mise à jour de l'UI. En cas de mort, une animation se joue avant la destruction.
- **Animations** : toutes les actions sont liées à un `Animator` avec des états dédiés (`idle`, `run`, `attack`, `hurt`, `death`).

Cette IA remplit parfaitement son rôle : elle permet une interaction fluide avec le joueur tout en restant légère sur le plan des performances.

3.3 Le multijoueur

3.3.1 Évolution du concept

Au départ, notre approche du multijoueur reposait sur une expérience coopérative classique : les deux joueurs devaient unir leurs forces pour vaincre une base ennemie contrôlée par l'IA, en partageant une **économie commune**, un **ensemble d'unités identiques** ainsi que le même **espace de déploiement**. Cette configuration encourageait fortement la coordination, chaque décision ayant un impact direct sur son allié.

Cependant, après plusieurs phases de test et de réflexion, ainsi que des échanges avec d'autres ingénieurs et développeurs, nous avons choisi de faire évoluer cette base vers un modèle plus moderne et flexible. Nous avons alors décidé d'implémenter le multijoueur à l'aide de **Photon PUN2**, une solution adaptée aux jeux en temps réel et parfaitement intégrée à Unity. Ce choix technique nous a permis d'établir une architecture en ligne solide, tout en séparant clairement les responsabilités entre les joueurs.

Ce choix s'est avéré pertinent au vu des nombreux avantages offerts par cette technologie :

- **Avantages**
  - Intégration facile avec Unity grâce à une documentation riche et accessible ;
  - Hébergement cloud sans besoin de gérer un serveur dédié, permettant de se concentrer pleinement sur le gameplay ;
  - Système mature, avec une gestion efficace de la synchronisation en temps réel (via PUN ou Fusion) ;
  - Compatibilité multiplateforme (PC, mobile, consoles, VR), ouvrant la voie à de futures évolutions.
- **Inconvénients**
  - Dépendance au cloud Photon, limitant la possibilité d'un hébergement personnalisé ;
  - Coût potentiel : l'offre gratuite devient payante au-delà de 20 utilisateurs connectés simultanément.

3.3.2 Concept final retenu

Nous avons finalement opté pour un système dans lequel chaque joueur est **entièrement autonome**. Contrairement à notre idée initiale, cette nouvelle approche offre une séparation totale des éléments de jeu. Plus précisément :

— chaque joueur dispose de son **propre ensemble d'unités**, différent de celui de son coéquipier ;

— chacun gère sa **propre économie** (ressources, revenus, dépenses) ;

— les décisions stratégiques sont prises de manière **indépendante**, sans contrainte directe imposée par l'autre joueur.

Aucune ressource n'est partagée, et la coopération repose uniquement sur une volonté commune de synchroniser ses actions pour vaincre l'adversaire contrôlé par l'IA. Cette configuration encourage une **véritable synergie tactique**, tout en respectant la liberté individuelle de chaque joueur.

C'est Raphaël qui s'est occupé de la partie multijoueur du projet. Il a mis en place l'architecture réseau avec Photon et s'est chargé d'adapter le prototype solo existant pour qu'il soit compatible avec un mode multijoueur. Ce travail a demandé une révision en profondeur de plusieurs éléments du jeu, avec l'objectif de préserver la cohérence entre les deux modes.

3.3.3 Implémentation technique

Grâce à **Photon PUN2**, Raphaël a mis en place un système de **Rooms** permettant la création, la recherche et la jointure de salons via les fonctions `PhotonNetwork.CreateRoom`, `JoinRoom`, et `JoinRandomRoom`. Une **salle d'attente interactive** avec minijeu a également été intégrée pour permettre au premier joueur de patienter de manière ludique en attendant son coéquipier.

3.3.4 Synchronisation réseau

Pour assurer une expérience fluide et cohérente entre les deux joueurs, les éléments suivants ont été synchronisés à travers le réseau :

— `PhotonNetwork.Instantiate` pour les unités et tourelles contrôlées par l'IA ;

— `PhotonTransformView` pour les mouvements et positions ;

— `PhotonAnimatorView` pour les animations (attaques, déplacements, morts) ;

— `PhotonView` pour le suivi de l'état des entités (barres de vie, effets, statuts) ;

— `PhotonView.RPC()` pour déclencher des effets ponctuels ou interactions spécifiques ;

— `PhotonNetwork.LoadLevel` pour assurer un chargement de scène synchronisé entre les deux clients.

Chaque entité essentielle est associée à un composant `PhotonView`, garantissant ainsi la mise à jour de son état sur l'ensemble des clients connectés.

3.3.5 Conclusion

L'intégration de **Photon PUN2** nous a permis de proposer une expérience multijoueur **réactive**, **stable** et **libre**, respectant les exigences d'un jeu en temps réel tout en assurant une synchronisation robuste entre les joueurs. Ce système donne à chaque joueur un contrôle total sur sa stratégie, ses décisions économiques et sa manière d'aborder la partie.

Plutôt que d'imposer des contraintes artificielles pour forcer la coopération, nous avons misé sur une **coopération volontaire et stratégique**, renforcée par la complémentarité des rôles. Ce choix permet d'adapter le gameplay à différents profils de joueurs et styles de jeu, tout en conservant une dynamique engageante. Grâce à cette flexibilité et à la stabilité offerte par Photon, l'expérience multijoueur devient à la fois **immersive**, **modulaire** et **évolutive**, prête à accueillir de futures extensions comme des modes compétitifs ou des campagnes coopératives avancées.

3.3.6 La scène de chargement

Entièrement développée par **Norah**, la scène de chargement multijoueur a été conçue pour offrir une expérience engageante dès l'attente des joueurs, loin d'un simple écran statique.

Lorsque le premier joueur se connecte en multijoueur, il est automatiquement dirigé vers cette scène d'attente, en attendant qu'un second joueur rejoigne la partie. Il s'agit d'une carte 2D au style visuel soigné, inspirée de la première carte du jeu où Célestin gouvernait tranquillement. L'ambiance y est paisible, naturelle et médiévale, avec des éléments animés comme des montagnes mouvantes, des nuages, un moulin en rotation, une charrette, des fleurs, des buissons et de l'herbe.

Un mini-jeu intégré. Plutôt que de laisser le joueur seul dans un espace vide, nous avons opté pour un mini-jeu inspiré des publicités de jeux mobiles, visant à rendre l'attente plus agréable et ludique. Le joueur incarne un petit chevalier en pixel art, dans une confrontation contre le *Seigneur des Bugs*.

Ce dernier est un personnage violet plus grand, volontairement en décalage visuel avec le décor : plus pixelisé, avec des effets de glitch et de téléportation. Il peut apparaître aléatoirement à environ 10 pas du joueur, se déplacer rapidement et se téléporter pour anticiper ses mouvements — un comportement directement inspiré de la "Mort" dans le film *Le Chat Potté 2*. Il possède également une animation d'attaque à longue portée. Pour équilibrer ce mini-jeu d'attente, le joueur dispose de 5 points de vie et le Seigneur des Bugs de 3. En cas de défaite de l'un des deux, il réapparaît immédiatement. Il ne s'agit donc pas d'une phase décisive du jeu, mais d'une manière ludique de faire patienter les joueurs avant le lancement de la vraie partie.

Un comportement plus passif de l'IA est également envisagé : si le joueur reste inactif, le Seigneur des Bugs ne l'attaque pas, se contentant de le suivre. Cela permet à ceux qui ne souhaitent pas participer activement au mini-jeu d'attendre sans pression.

Interface utilisateur dédiée. L'interface de cette scène reprend les éléments habituels, tout en les adaptant au contexte :

— En haut à gauche : la vie du Seigneur des Bugs. L'affichage de ces textes est volontairement altéré pour donner un effet "glitch", renforçant l'identité du mini-jeu.

— En haut au centre : le nom du serveur en cours et la mention `En attente de joueur`, pour bien distinguer cette scène de la partie principale.

— En haut à droite : les paramètres (volume, plein écran) sont accessibles, avec deux boutons :

  — **Retour** : permet de revenir dans la scène de mini-jeu si l'on s'est rendu dans les paramètres.
  — **Menu** : retourne au menu principal.

— En bas ou à un emplacement visible : le compteur de joueurs connectés (`1/2`, `2/2`), affiché en jaune tant que tous les joueurs ne sont pas prêts, et passant au vert dès que les deux sont connectés.

Cette scène de chargement est ainsi bien plus qu'un simple écran d'attente : elle illustre notre volonté de proposer une expérience riche et cohérente du début à la fin, en transformant chaque instant de jeu, même passif, en un moment plaisant et interactif.

3.4 L'interface utilisateur



FIGURE 2 – *
Illustration de l'interface en jeu

L'interface utilisateur est un élément clé de notre jeu, permettant au joueur de disposer des informations essentielles et d'interagir facilement avec le système de jeu. Elle a été pensée pour être à la fois fonctionnelle, immersive et intuitive. Voici les différents composants de cette interface, accompagnés de leurs contributeurs respectifs :

3.4.1 Barre de Vie

Barre de vie et indicateurs visuels

Développée par **Foucauld**, cette interface affiche plusieurs barres de vie à l'écran, représentant la santé des entités clés : le **Château**, la **les Unités** ou encore **les Tours**

Les barres de vie sont positionnées de manière stratégique :
— En haut au centre de l'écran pour le château.
— Au-dessus des unités, pour permettre un suivi précis en temps réel.

L'objectif du joueur est de réduire la barre de vie du château à **zéro** avant la fin du temps imparti, afin de remporter la partie. Ces barres constituent donc un indicateur central de la progression du joueur.

Tout ce système a été développé pour fonctionner **en multijoueur** : chaque changement d'état (dégâts, mort, affichage) est entièrement **synchronisé entre les joueurs via Photon PUN** pour garantir une expérience cohérente.

Les barres de vie utilisent également un **dégradé de couleurs dynamique**, allant du vert (plein) au rouge (critiquement bas), qui évolue en temps réel en fonction des points de vie restants. Ce dégradé est lui aussi parfaitement **synchronisé avec les dégâts reçus**, de manière à refléter avec précision l'état des entités pour tous les joueurs simultanément.

Ce système améliore fortement la lisibilité de l'action en cours et renforce l'immersion, tout en permettant une prise de décision rapide et partagée dans le cadre du jeu coopératif.

3.4.2 Temps et Argent

Cette partie a été conçue par **Norah**. Étant donné que le jeu repose sur un système de manches, deux compteurs sont affichés à gauche de l'écran :

— Le temps **total** du jeu : durée limite pour terminer la carte.
— Le temps **par manche** : période pendant laquelle le joueur peut acheter et placer ses unités.

À la fin de chaque manche, les unités sont déployées automatiquement et le joueur gagne de la monnaie, affichée en temps réel via l'interface. Ce système fonctionne en multijoueur grâce au script `GameManager`, qui synchronise les temps et l'argent pour tous les joueurs.

3.4.3 Acheter des unités

Initialement développé par **Julien**, l'achat d'unités se faisait via un inventaire classique. Après discussion en groupe, une nouvelle approche a été mise en place : une barre d'accès rapide inspirée des jeux comme *Minecraft*, développée par **Julien et Norah**.

Les unités sélectionnées depuis le menu apparaissent directement dans cette barre une fois en jeu. Chaque unité y est représentée avec :

— Une image,
— Son nom,
— Son prix.

Ce système permet un accès rapide et intuitif aux unités pendant les phases de placement.

3.4.4 Les paramètres

L'interface des paramètres est issue d'un premier travail de **Norah**, puis adaptée au multijoueur par **Raphaël** afin que chaque joueur puisse avoir ses propres préférences. Les paramètres sont persistants grâce à un système de sauvegarde, même lors de changements de scène. On y trouve :

— Un **slider** pour ajuster le volume,
— Un **toggle** pour activer ou désactiver le plein écran,
— Un bouton **Retour** pour revenir au jeu,
— Un bouton **Menu** pour retourner au menu principal.

Ces options offrent une meilleure compréhension des combats en cours et renforcent l'immersion.

3.4.5 Le tutoriel

Conçu par **Norah**, le tutoriel aide les joueurs, notamment les moins expérimentés avec les tower defense. Il est accessible via un bouton et se compose de plusieurs pages contenant :

3.4.6 Système de caméra

Dans le cadre du développement du projet *TowerAttack*, Raphaël a mis en place un système de caméra avancé permettant une navigation fluide, intuitive et personnalisable dans l'environnement 3D du jeu. Ce système repose sur les éléments suivants :

La caméra libre permet au joueur ou développeur de se déplacer librement dans la scène en utilisant les touches `W`, `A`, `S`, `D` pour les déplacements horizontaux, `Espace` pour monter, et `Shift` pour descendre.

La rotation est contrôlée par la souris, avec verrouillage ou déverrouillage du curseur via une touche dédiée la touche `E`. Ce système est particulièrement utile pour observer le champ de bataille, tester le level design, ou réaliser des prises de vue dynamiques.

Raphaël a intégré un système de points de téléportation prédéfinis dans la scène. Six points peuvent être assignés manuellement (ex. : Spawn, Début, Milieu, Fin, etc.). En appuyant sur les touches numériques `1` à `6`, la caméra est instantanément téléportée au point correspondant, avec position et rotation conservées. Cela permet un accès rapide à des zones spécifiques de la carte.

La FreeCam permet de basculer entre un mode "verrouillé" (curseur invisible, rotation libre) et un mode "libre" (curseur visible, interaction UI). Un zoom dynamique a été ajouté via le champ de vision (`fieldOfView`), avec une plage paramétrable entre un minimum et un maximum de `0` à `120`).

Toutes les caméras du projet sont centralisées via un gestionnaire dédié, `CamerasManager` qui maintient une liste à jour des caméras disponibles et garantit qu'une seule caméra est active à la fois. Le `SetCamera()` permet de basculer proprement entre plusieurs caméras (ex. FreeCam, caméras fixes, caméras d'unités), en désactivant les autres et en désactivant leur `AudioListener` pour éviter les conflits.

Lors de l'instanciation d'unités (via prefab), Raphaël a observé que les caméras enfants des unités pouvaient prendre le contrôle automatiquement. Pour éviter cela, un désactivateur automatique de caméras internes a été mis en place dès l'instanciation, garantissant que la vue reste contrôlée par le système principal.

Tous les scripts qui impliquent des contrôles caméra (comme le script du joueur ou de la FreeCam) sont conditionnés à la caméra active. Cela évite tout conflit ou comportement simultané, notamment dans le contexte multijoueur ou d'observation parallèle.

Résumé

- Une caméra libre fluide et intuitive
- Un système de téléportation rapide entre points stratégiques
- Un verrouillage du curseur propre pour basculer entre interaction et navigation
- Une gestion centralisée et propre des caméras
- Une prévention automatique du switch caméra involontaire au spawn
- Un socle stable pour des extensions futures (vue unité, cinématique, replay, etc.)

3.4.7 Les images de fin

Cette interface finale a également été développée par **Norah**.

À la fin d'une partie, une fenêtre contextuelle (ou *pop-up*) s'affiche, avec un contenu différent selon le résultat :

- **En cas de victoire** :
  - Une affiche verte avec l'inscription « Victoire »,
  - Un gain de **+500** monnaie,
  - Trois boutons : *Rejouer*, *Menu*, *Quitter*.
- **En cas de défaite** :
  - Une affiche rouge avec l'inscription « Défaite »,
  - Un gain de **+200** monnaie,
  - Les mêmes boutons que pour la victoire.

En mode **multijoueur**, tous les joueurs doivent cliquer sur *Rejouer* pour relancer une partie.

Le contenu des pop-ups peut également varier selon que le joueur soit en mode **Histoire** ou en mode **Campagne**, notamment en cas de victoire.

Dans le mode Histoire et en Multijoueur, les joueurs enchaînent les **trois premières cartes**. Lorsqu'un joueur gagne, il peut choisir de quitter via le pop-up de victoire. Cependant, s'il ne clique pas, il sera automatiquement téléporté vers la carte suivante au bout de **7 secondes**, et ce de manière synchronisée avec les autres joueurs.

S'ils parviennent à remporter la **troisième carte**, appelée la « map Finale », les joueurs auront alors la possibilité de :

- **Libérer Célestin de sa cage**,
- **Gagner un niveau**,
- **Acheter toutes les unités du jeu**.

Ce système de progression a été développé par **Raphaël**.

3.4.7 Conclusion

Cette interface utilisateur a été conçue pour équilibrer accessibilité et profondeur, tout en s'intégrant naturellement dans l'univers graphique et sonore du jeu. Chaque élément a été testé pour garantir sa clarté et son utilité pendant les phases actives de gameplay.

3.5 Les cartes

Nous avions initialement prévu de créer quatre cartes représentant différentes époques et lieux à travers lesquels le héros progresse pour libérer Célestin. Ce choix a été maintenu dans l'esprit, bien que le rythme de production ait été ajusté à la suite de notre première soutenance.

Dans le scénario du jeu, le héros traverse le temps et l'espace, de l'époque médiévale à un futur lointain, afin d'affronter le Seigneur des Bugs et délivrer le roi Célestin.

La conception et le développement des cartes ont été réalisés par **Norah**, qui a su créer trois environnements principaux, chacun doté d'une ambiance visuelle distincte, de défis spécifiques et d'une progression scénaristique claire. Ces cartes servent de toile de fond immersive aux mécaniques de jeu, tout en renforçant la narration globale.

Carte test et carte 1 – Thème médiéval / Forêt



FIGURE 3 – Vue générale de la carte à thème médiéval du projet Tower Attack

La première phase a été la création d'une carte test, qui nous a permis d'apprendre à structurer une carte de jeu :

— Optimisation de la carte,
— Réalisme des décors,
— Esthétique globale,
— Brume et ciel évolutifs.

Nous avons enrichi le décor avec des fleurs, de l'herbe, des cailloux, un pont, de l'eau, et divers objets naturels. Cependant, nous avons rencontré des difficultés techniques, notamment avec les textures (affichage rose des arbres) et la modélisation de l'eau, qui apparaissait en plaques multiples peu réalistes.

La première carte, de type *Low Poly*, comprend :

— un **village médiéval** détaillé avec une forge, une boulangerie, une meunerie, une taverne, une poissonnerie, un coin d'entraînement, une boucherie-charcuterie et un magasin de légumes ;

— une **forêt** traversée par un chemin menant à une roche en forme de dragon contenant une rune magique.

Nous avons résolu les problèmes rencontrés :

— Raccordement correct des matériaux pour les assets,

— Ajout d'éléments un par un pour plus de précision,

— Réalisation d'un plan d'eau par étirement d'un seul objet,

— Correction des contrôles caméra/direction après de nombreux tests.

Carte 2 – Thème urbain / Ville moderne



FIGURE 4 – Vue générale de la carte à thème urbain du projet Tower Attack

Cette carte se situe dans un univers plus contemporain. L'ambiance visuelle est marquée par un ciel rouge-orangé évoquant un coucher de soleil. Elle marque une rupture avec le style médiéval des cartes précédentes, tout en s'inscrivant dans la progression narrative du jeu. Le design urbain, les véhicules modernes et les bâtiments détaillés participent à créer un contraste visuel fort avec les environnements plus anciens.

— Présence de bâtiments modernes ;

— Route principale traversée par des véhicules ;

— Ambiance urbaine dense avec une esthétique de fin de journée ;

— Palette de couleurs chaudes dominante.

Ce changement d'époque illustre la traversée temporelle du héros, et prépare le joueur à une transition progressive vers un univers plus sombre et technologique.

Carte 3 – Thème futuriste / Monde nocturne

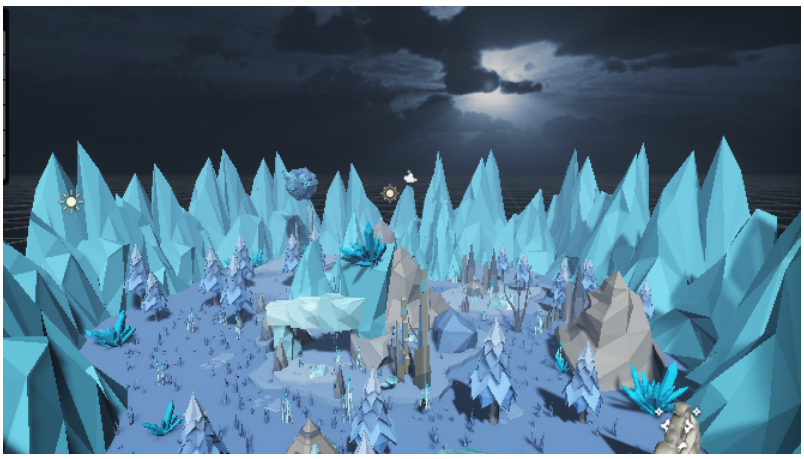


FIGURE 5 – Vue générale de la carte finale à thème futuriste

La dernière carte du jeu plonge le joueur dans une ambiance bleutée et nocturne. Le style visuel est résolument futuriste, avec des structures technologiques et des effets lumineux accentuant l'atmosphère mystérieuse. Cette carte incarne la dernière étape du parcours narratif, tant sur le plan esthétique que symbolique.

- Ambiance sombre et froide, dominée par des tons bleus ;
- Présence de lumières artificielles et de structures high-tech ;
- Objectif final : atteindre le totem qui retient Célestin prisonnier dans une cage et absorbe son pouvoir de création ;
- Sentiment d'aboutissement, de tension dramatique et de défi final.

Ce thème renforce l'évolution temporelle du jeu et crée un climax narratif et visuel fort. Le cycle **médiéval → moderne → futuriste** se trouve ainsi complété, offrant au joueur une expérience immersive à travers les époques et les épreuves.

3.6 Choix musicaux et ambiance sonore

La bande-son, Développée par **Norah**, joue un rôle fondamental dans la création de l'ambiance et l'immersion du joueur. Pour notre projet, nous avons sélectionné des morceaux **libres de droits** qui correspondent à différentes phases de l'expérience ludique, tout en renforçant les émotions et dynamiques propres à chaque scène.

Direction artistique

Nous avons privilégié une ambiance **épique et immersive**, adaptée à l'univers stratégique et narratif du jeu. Les musiques choisies partagent des tonalités orchestrales, parfois dramatiques ou mystérieuses, en accord avec les moments clés de la progression du joueur.

Liste des musiques utilisées

- **"Doors Elevator" – APM Music**
  Ce morceau est utilisé dans la **scène de chargement multijoueur**. Il s'agit de la musique d'attente emblématique du jeu *Roblox Doors*, jouée dans l'ascenseur au début de la partie. Son ambiance légèrement inquiétante, mais calme, correspond parfaitement à notre carte d'attente. Elle renforce l'idée que quelque chose se prépare, sans être anxiogène, tout en offrant un fond sonore engageant durant la phase de connexion.
- **"THE IMPOSSIBLE" – Plume**
  Un morceau orchestral intense, utilisé lors de phases de jeu stratégiques ou de combat. Il apporte une sensation de tension maîtrisée, évoquant un affrontement à venir ou une mission périlleuse.
- **"Viking" – Louis Brunet**
  Comme son nom l'indique, cette musique s'inspire de sonorités nordiques, avec des percussions puissantes et une ambiance guerrière. Elle accompagne des moments où le joueur avance dans un environnement rude, renforçant l'aspect épique de certaines cartes.
- **"Behind Enemy Lines" – Plume**
  Ce morceau évoque l'infiltration, la discrétion et le danger. Il est utilisé dans des séquences plus calmes ou mystérieuses du jeu, où le joueur doit se montrer stratégique. L'atmosphère musicale aide à créer une tension discrète mais constante.
- **"Puttin's Lullaby" – Plume**
  Plus doux, ce morceau sert à ponctuer des moments narratifs ou plus légers. Il permet de marquer des transitions dans le rythme de jeu, ou de conclure une séquence sur une note plus émotionnelle ou poétique.

Choix techniques

Toutes les musiques sont diffusées à l'aide du système audio de notre moteur de jeu, avec un volume ajusté en fonction du contexte. Le joueur peut par ailleurs régler le volume de la musique via les **paramètres du menu principal**, pour une meilleure personnalisation de son expérience.

Ces musiques enrichissent l'univers de *Tower Attack* en renforçant l'identité émotionnelle de chaque scène, tout en respectant les contraintes techniques et légales liées à l'utilisation de contenus libres de droits.

3.7 Communication

3.7.1 L'intérieur de l'entreprise

Au sein de notre entreprise, nous communiquons principalement via un serveur Discord, qui nous permet de rester en contact régulier grâce aux appels vocaux. Nous avons également mis en place plusieurs salons textuels dédiés, ce qui nous aide à nous organiser efficacement sur différents sujets. Centraliser toutes les informations dans un seul espace nous évite de devoir les chercher dans des discussions dispersées et rends notre travail plus rapidement accessible.

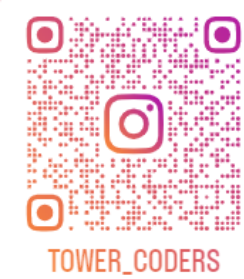
En complément, nous utilisons Trello, une plateforme qui nous permet de partager notre progression sur les différentes tâches. Nous y avons structuré notre travail en trois catégories : À faire, À améliorer, et Fini. Cette organisation nous aide à suivre l'avancement des projets de manière claire et collaborative.

Nous partageons également nos codes ainsi que tous les documents annexes grâce à Github, plateforme sur laquelle nous avons rencontré de nombreux problèmes, que nous avons réussi à résoudre rendant son utilisation plus efficace.

Nous avons également pensé à entamer la communication assez tôt en créant un site web et un compte Instagram. Pour un aspect plus professionnel, nous avons également créé une adresse Gmail spécifique à notre entreprise : tower.coders.epita@gmail.co

3.7.2 Instagram

Le compte Instagram mit à jour par **Norah**, nous sert de vitrine pour notre jeu vidéo. Nous y partageons notre progression et des informations clés. Afin de toucher un public international, nous publions nos contenus en Anglais et en Français, en y ajoutant une touche d'humour pour élargir notre audience. Pour obtenir des visuels attrayants, nous avons passé plusieurs après-midi dans Rennes à photographier Célestin, afin de construire une identité visuelle autour de ce personnage. Aujourd'hui, notre compte a déjà plus de 52 abonnés, et ce n'est que le début.



3.7.3 Site internet

Afin de faire connaître notre entreprise et notre projet en cours, **Julien** a créé notre site internet. Après discussion, nous avons décidé d'utiliser la plateforme spécialisée Odoo, qui propose un service de création de sites internet intuitif et qui permet de développer des sites très professionnels.

Celle-ci met davantage l'accent sur notre entreprise et crée un lien avec notre communauté en présentant le projet Tower Attack. Elle met également en avant les membres de l'équipe, avec leur biographie et leur rôle au sein de l'entreprise.

Notre site, conçu pour être dynamique et interactif, propose une navigation intuitive grâce à plusieurs boutons intéractifs permettant d'accéder facilement aux différentes rubriques.

Nous avons construit notre site en 3 blocs principaux : Page d'Accueil, Contactez-nous et les News de Tower Attack.

Notre site est régulièrement mis à jour et alimenté en contenu pour que nos futurs joueurs soient au courant de toutes nos nouveautés mais aussi se sentent plus proche de notre entreprise et de notre projet en cours.

À terme, les utilisateurs pourront télécharger nos jeux depuis notre site via une rubrique dédiée. Il accueillera peut-être un jour une boutique en ligne de produits dérivés en tous genres.

Nous avons fait également une documentation des différentes unités qui sont prévus d'être implémenté.



3.8 Conclusion

3.8.1 Tableau de Gantt



3.8.2 Avancement

Une fois notre première carte fonctionnelle, cela a marqué un tournant dans le développement. À partir de là, nous avons pu enrichir notre jeu de nombreuses fonctionnalités : un système de succès pour encourager le joueur, un mini-jeu intégré lors du temps de chargement pour le rendre plus interactif, un menu de paramètres personnalisables, et même un magasin intégré dans l'interface principale. Cet avancement nous a permis de pousser plus loin nos ambitions, jusqu'à la création de deux nouvelles cartes jouables. Bien que certains petits bugs persistent, nous sommes fières d'avoir su nous dépasser pour intégrer autant de nouvelles idées dans un temps limité.

L'intelligence artificielle, au cœur du gameplay, a quant à elle nécessité des ajustements constants tout au long du développement, jusqu'aux dernières étapes du projet. Chaque itération nous a permis d'améliorer le comportement des unités et d'enrichir l'expérience de jeu.

Par ailleurs, grâce aux cours de méthodologie, nous avons appris à aborder ce projet de manière plus réaliste et professionnelle. Cela nous a permis de créer une véritable identité visuelle : choix cohérents de couleurs, conception d'un logo, élaboration d'un style graphique propre à notre projet. Ce travail sur l'image et la présentation nous a aussi aidées à progresser sur notre aisance à l'oral et dans notre capacité à défendre un projet devant un public. Ce fut une expérience très enrichissante.

3.8.3 Ressenti du projet

> Foucauld Bracq :

Globalement, le projet s'est bien passé. On a réussi à aller au bout de quelque chose de concret et fonctionnel, ce qui est déjà une belle satisfaction. Il y a eu un peu de rush sur la fin, comme c'est souvent le cas dans ce genre de projet, mais on a tenu bon et réussi à intégrer un maximum de fonctionnalités.

Au début, on a quand même ressenti une certaine disparité dans la répartition du travail, j'ai pris plus les devants pendant que d'autres avaient plus de mal à trouver leur place. Mais avec le temps, les rôles se sont mieux équilibrés et chacun a pu contribuer à sa manière.

L'apprentissage de Git a été un vrai tournant : ça nous a permis de mieux collaborer, d'éviter les conflits de fichiers, et de travailler chacun de notre côté sans bloquer les autres. C'était un vrai plus pour avancer efficacement à plusieurs.

En résumé, ce projet m'a permis de mieux comprendre le travail en équipe, de gagner en autonomie, et de prendre en main des outils de développement indispensables. Malgré les moments de stress, je garde un très bon souvenir de cette expérience.

> Norah Beillevert :

Au début de l'année, je ne pensais pas être capable de réaliser un projet aussi ambitieux. Je partais sans vraie expérience dans le développement de jeux, avec pas mal d'appréhensions. Petit à petit, j'ai appris à maîtriser des outils, à comprendre des mécaniques de jeu, à coder des éléments complexes... Mais le plus difficile n'a pas été que technique.

Il y a eu des moments très durs moralement. J'ai eu peur, à plusieurs reprises, que le travail repose uniquement sur quelques personnes du groupe. Et ce qui a été encore plus difficile, c'est de voir certaines personnes critiquer notre travail, le dévaloriser, voire l'attribuer à d'autres. Ça m'a vraiment affectée, car j'y avais mis beaucoup de temps, d'énergie et de cœur.

Malgré tout, j'ai tenu bon. Ce projet m'a permis de découvrir que j'étais capable de beaucoup plus que ce que je pensais. J'ai pu toucher à différents domaines : interface utilisateur, IA, multijoueur... et surtout, j'ai appris à m'accrocher, à garder confiance en moi même quand c'était compliqué.

Aujourd'hui, je suis fière du travail accompli. Et malgré les difficultés, cette expérience m'a donné envie de continuer. J'ai même envie de créer un nouveau projet en 2D, à mon rythme, pour mettre en pratique ce que j'ai appris dans un cadre plus serein. Ce projet a été une épreuve, mais aussi une grande victoire personnelle.

> Raphaël Dréau :

J'ai vraiment pris plaisir à travailler sur ce projet. J'étais bien entouré, avec une équipe avec qui j'ai aimé collaborer, et ça a clairement rendu le travail plus motivant et agréable au quotidien. Je suis assez content du résultat final, surtout parce qu'il reflète bien l'investissement et les efforts qu'on y a mis tous ensemble.

Je suis fier de ce que j'ai pu faire, aussi bien sur le plan technique que personnel. Ce projet m'a permis de progresser, de mieux bosser en équipe, et de gagner en confiance. Au final, je suis vraiment satisfait de ce qu'on a réussi à construire avec mes collègues.

> Thomas Vigneau :

J'ai beaucoup apprécié travailler sur ce projet. Il m'a permis de mettre en pratique ce que j'ai appris, tout en explorant de nouvelles idées. Le fait d'évoluer dans une équipe motivée et bienveillante a rendu l'expérience encore plus enrichissante et agréable au quotidien.

Ce projet m'a poussé à me dépasser, à mieux collaborer et à prendre plus d'initiatives. J'en ressors avec de nouvelles compétences, mais aussi avec la satisfaction d'avoir contribué à quelque chose de concret et abouti. Je suis fier du travail accompli, aussi bien individuellement qu'en équipe.

> Julien Jollivet :

Ce projet a été une expérience extrêmement enrichissante pour moi. En début d'année, je n'imaginais pas avoir les compétences nécessaires pour réaliser un jeu aussi complet. J'ai pu développer de nouvelles connaissances, aussi bien sur le plan technique qu'humain.

Travailler en équipe m'a permis de mieux comprendre l'importance de la collaboration, de la communication, et de l'écoute dans un projet de groupe. Nous avons réussi à créer une bonne dynamique, et l'ambiance de travail a toujours été agréable, ce qui a largement contribué à notre motivation et à la progression du projet.

J'ai également eu l'opportunité de maîtriser de nouveaux outils que je ne connaissais pas auparavant, que ce soit pour la gestion de projet, le développement en réseau, ou encore l'implémentation de l'intelligence artificielle. Cette diversité de tâches m'a vraiment permis d'élargir mes compétences.

Au final, je suis très fier de ce que nous avons accompli ensemble. Ce projet m'a donné envie de continuer à créer d'autres jeux ou applications, même en dehors du cadre scolaire.

3.8.4 Compétences et apprentissages

Tout au long de ce projet, nous avons eu l'occasion d'enrichir nos compétences sur plusieurs plans. Le choix de travailler avec Unity et le langage C nous a permis de plonger concrètement dans le développement de jeu vidéo, en allant bien au-delà de simples notions théoriques. Chaque fonctionnalité du jeu a été une occasion d'apprendre : implémenter un système de vague, créer une interface intuitive, synchroniser un mode multijoueur avec Photon, ou encore gérer des animations dynamiques en temps réel.

La gestion de projet a également été un aspect formateur essentiel. L'utilisation d'un outil de versionnage comme Git, que nous avons appris à maîtriser au fil du temps, nous a permis de travailler efficacement à plusieurs sur le même code sans conflits majeurs. Cela nous a aussi sensibilisés aux bonnes pratiques de développement collaboratif : faire des commits réguliers, rédiger des messages clairs, et résoudre collectivement les problèmes rencontrés.

Au-delà du code, le projet nous a aussi confrontés à des défis liés au game design et à l'ergonomie. Définir un rythme de jeu satisfaisant, équilibrer la difficulté des ennemis, choisir une palette de couleurs cohérente, concevoir des menus lisibles ou décider de la meilleure façon de guider le joueur lors de sa première partie sont autant de problématiques que nous avons dû explorer et résoudre ensemble. Ces aspects, souvent négligés, nous ont permis de mieux comprendre l'importance de penser le jeu comme un tout, mêlant technique, esthétique et expérience utilisateur.

Sur le plan humain, ce projet a été une véritable école du travail en équipe. Nous avons appris à répartir les tâches selon les forces de chacun, à nous entraider, à faire face aux retards et aux imprévus, à adapter nos ambitions aux contraintes du temps et de la technique. Il a parfois fallu faire des compromis, revenir en arrière ou revoir nos priorités, mais c'est précisément dans ces moments-là que nous avons le plus appris.

Enfin, chacun a pu développer sa propre polyvalence : certains se sont spécialisés

dans l'IA, d'autres dans l'interface ou l'animation, mais tous ont été amenés à explorer plusieurs domaines, parfois très éloignés de leurs compétences initiales. Cette ouverture nous a permis de mieux comprendre les différents métiers liés au développement de jeux vidéo, et de gagner en autonomie sur des outils complexes comme Unity, Blender, Mixamo ou Photon PUN.

En résumé, ce projet nous a permis d'acquérir bien plus que des lignes de code ou un jeu jouable : il nous a donné des méthodes, des outils, de la confiance, et surtout une véritable culture du travail collaboratif. Ce sont là des acquis durables, qui nous serviront dans bien d'autres projets à venir.

3.8.5 Conclusion de l'entreprise

Chez Tower Coders, ce projet de jeu vidéo a marqué une étape majeure dans notre aventure de développeurs. Il représente l'aboutissement de plusieurs mois de travail, de recherches, de tests, et surtout de passion. Plus qu'un simple exercice technique, ce jeu a été l'occasion pour notre équipe de consolider ses compétences, d'explorer de nouveaux domaines, et de relever ensemble un défi ambitieux.

Dès le départ, notre objectif était clair : créer une expérience de jeu stratégique et accessible, dans laquelle le joueur invoque des unités pour attaquer une base ennemie dans un univers dynamique. Ce concept de base a évolué au fil du développement pour intégrer des mécaniques avancées comme un système de boutique, une gestion de la monnaie, un enchaînement de manches, des barres de vie, des cartes multiples, un mini-jeu bonus, une IA pour les unités, un tutoriel pour les nouveaux joueurs, ainsi qu'un mode multijoueur fluide grâce à l'intégration de Photon.

Mais au-delà des fonctionnalités, ce projet a surtout été une formidable opportunité de collaboration. Nous avons appris à travailler en équipe, à nous organiser, à prendre des décisions ensemble, à résoudre les problèmes de manière collective, dans un climat de bonne humeur et d'entraide. Ce sont ces qualités humaines, autant que les compétences techniques, qui ont permis à Tower Coders d'avancer sereinement et efficacement.

Ce projet a également permis à chaque membre de l'équipe de sortir de sa zone de confort. Beaucoup d'entre nous sont partis de zéro au début de l'année, sans connaissances en C#, en Unity ou en conception de jeu. Aujourd'hui, nous avons exploré l'interface utilisateur, les mécaniques de gameplay, l'IA, le multijoueur, la gestion de sauvegarde, et bien d'autres aspects essentiels du développement. C'est une progression dont nous sommes fiers.

Cette première création signe le début d'un parcours que nous souhaitons poursuivre. Forts de cette expérience, plusieurs membres de Tower Coders envisagent

déjà de continuer à développer des projets en dehors du cadre scolaire, comme Tower Attack ou d'autres projets 2D, pour expérimenter encore davantage et pousser plus loin notre créativité.

En conclusion, ce jeu n'est pas qu'un produit fini. Il est le reflet de tout ce que Tower Coders a appris, construit et partagé durant ces mois de travail. Et surtout, il incarne notre passion commune pour la création vidéoludique. Ce n'est que le début.